

MINI PROJECT – FINANCIAL DATA ANALYTICS

1. Aim

To clean, transform, store, and analyze financial transaction data using Python, MySQL, Power BI and Tableau to identify income and expense patterns, category-wise spending, monthly financial trends, and key financial performance indicators.

2. Problem Statement

Raw financial transaction data contains many records and may contain missing values, duplicate identifiers, inconsistent account labels, invalid dates and negative amounts. Manual inspection makes it difficult to identify income, expenses, spending categories, monthly performance, payment usage and high-value transactions.

3. Objectives

1. Load and inspect the transaction dataset.
2. Handle missing values and duplicate records.
3. Standardize dates, categories and transaction fields.
4. Create useful financial fields such as month, year, quarter and transaction type.
5. Store cleaned records in MySQL.
6. Analyze income, expenses, transaction amounts, merchants and payment-related fields.
7. Create Power BI DAX measures and interactive visuals.
8. Create Tableau calculated fields, parameters, filters and dashboard actions.
9. Interpret financial patterns and identify areas for monitoring.

4. Dataset Description

The selected financial dataset is a synthetic educational dataset. The repository documentation states that it contains 48,000 rows and fields including transaction ID, transaction date, customer ID, account type, transaction type, amount, currency, merchant and transaction status.

Field	Type	Purpose
transaction_id	Integer	Unique transaction identifier
transaction_date	Date	Date of transaction
customer_id	Integer	Customer reference
account_type	Text	Checking, Savings or Credit account
transaction_type	Text	Credit or Debit transaction
amount	Decimal	Transaction amount
currency	Text	Currency code
merchant	Text	Merchant name
transaction_status	Text	Transaction status

Verified source details: 48,000 rows were generated in the referenced educational project. The project documentation also records intentionally introduced dirty-data counts before cleaning, which are used below as source-supported audit results.

Issue	Documented count	Percentage
NULL customer_id	18,464	38.5%
NULL transaction_date	6,585	13.7%
Future date 2099-01-01	7,464	15.6%
Invalid account_type	7,217	15.0%
Negative amount	10,420	21.7%
Invalid status	11,301	23.5%
NULL merchant	7,965	16.6%

5. Tools and Technologies Used

Tool	Use in the project
Python	Data loading, cleaning, transformation and EDA
Pandas	DataFrame operations, grouping and aggregation
NumPy	Numerical operations
MySQL 8	Relational storage and SQL analysis
Power BI	Data model, DAX and interactive dashboard
Tableau	Calculated fields, parameters, filters and dashboard actions

6. System / Architecture

Financial Analytics – System Architecture

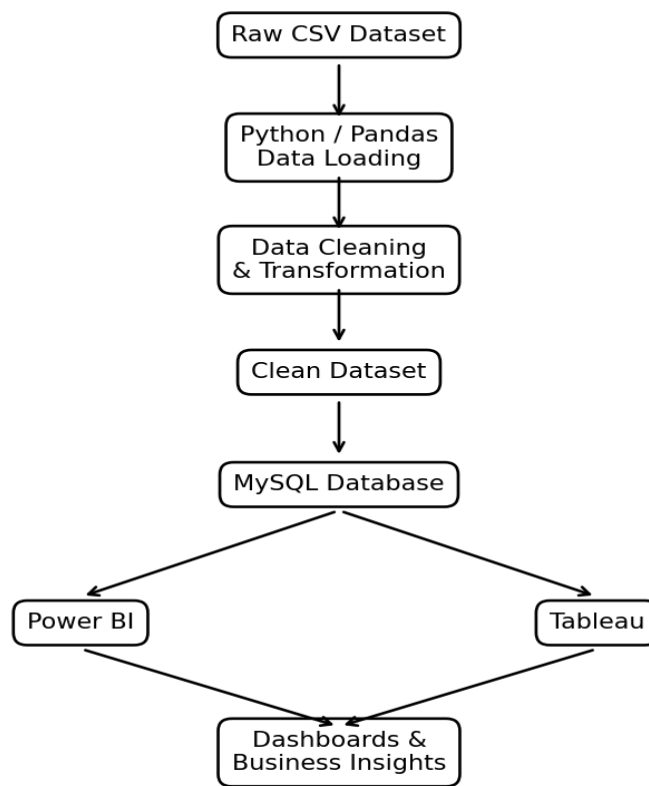


Figure: End-to-end implementation architecture.

7. Algorithm / Methodology

- 1.** Load dataset using Pandas.
- 2.** Inspect the dataset.
- 3.** Handle missing values.
- 4.** Remove duplicates.
- 5.** Correct data types.
- 6.** Standardize column names.
- 7.** Convert date fields.
- 8.** Create derived financial fields.
- 9.** Store cleaned data in MySQL.
- 10.** Connect MySQL with Power BI.
- 11.** Create DAX measures.
- 12.** Build interactive Power BI dashboard.
- 13.** Connect Tableau to the financial data.
- 14.** Create calculated fields and parameters.
- 15.** Build Tableau dashboard.
- 16.** Analyze and interpret results.
- 17.** End.

8. Python Data Loading

```
import pandas as pd
import numpy as np

df = pd.read_csv("Financial_Transactions_Clean.csv")
print(df.head())
print(df.shape)
print(df.dtypes)
```

The program reads the CSV file into a Pandas DataFrame and displays the first rows, shape and data types.

9. Python Data Cleaning

```
df.columns = (df.columns.str.strip().str.lower()
              .str.replace(" ", "_"))

print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# Standardize text fields
df["account_type"] = df["account_type"].astype(str).str.strip().str.title()
df["transaction_type"] = df["transaction_type"].astype(str).str.strip().str.title()
df["transaction_date"] = pd.to_datetime(df["transaction_date"], errors="coerce")
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")

# Keep valid financial records
df = df.drop_duplicates()
df = df[df["amount"].notna()]
```

The cleaning stage checks missing values and duplicates, converts numeric/date fields and standardizes categorical values.

10. Python Data Transformation

```
df["year"] = df["transaction_date"].dt.year
df["month"] = df["transaction_date"].dt.month
df["month_name"] = df["transaction_date"].dt.strftime("%B")
df["quarter"] = df["transaction_date"].dt.quarter

df["income"] = np.where(df["transaction_type"].eq("Credit"), df["amount"], 0)
df["expense"] = np.where(df["transaction_type"].eq("Debit"), df["amount"], 0)
df["profit"] = df["income"] - df["expense"]
df["profit_margin"] = np.where(df["income"] != 0,
                               df["profit"] / df["income"] * 100, 0)

df["amount_category"] = pd.cut(
    df["amount"], bins=[-np.inf, 500, 2000, 5000, np.inf],
    labels=["Low", "Medium", "High", "Very High"])

df.to_csv("financial_transactions_cleaned.csv", index=False)
print(df.head())
```

Derived fields are created for time-based analysis and dashboard measures. The exact formula is kept consistent across Python, SQL, DAX and Tableau.

11. Python Exploratory Analysis

```
print("Total transactions:", len(df))
print("Total income:", df["income"].sum())
print("Total expenses:", df["expense"].sum())
print("Net profit:", df["profit"].sum())
print("\nAverage transaction:", df["amount"].mean())
print("\nTransaction type summary:")
print(df.groupby("transaction_type")["amount"].agg(["count", "sum", "mean"]))
print("\nMerchant summary:")
print(df.groupby("merchant")["amount"].sum().sort_values(ascending=False).head(10))
print("\nMonthly summary:")
print(df.groupby(["year", "month"])[["income", "expense", "profit"]].sum())
```

These statements provide the main exploratory summaries used to validate the dataset before loading it into MySQL.

12. MySQL Database Creation

```
CREATE DATABASE IF NOT EXISTS financial_analytics;  
USE financial_analytics;
```

The database is created before the cleaned CSV is imported.

13. MySQL Table Creation

```
CREATE TABLE financial_transactions (  
    transaction_id INT,  
    transaction_date DATE,  
    customer_id INT,  
    account_type VARCHAR(30),  
    transaction_type VARCHAR(20),  
    amount DECIMAL(12,2),  
    currency VARCHAR(10),  
    merchant VARCHAR(100),  
    transaction_status VARCHAR(30),  
    year INT,  
    month INT,  
    month_name VARCHAR(20),  
    quarter INT,  
    income DECIMAL(12,2),  
    expense DECIMAL(12,2),  
    profit DECIMAL(12,2),  
    profit_margin DECIMAL(12,4),  
    amount_category VARCHAR(20)  
);
```

The table fields match the cleaned dataset and the derived fields used by the BI layer.

14. Loading Data into MySQL

```
LOAD DATA LOCAL INFILE 'financial_transactions_cleaned.csv'  
INTO TABLE financial_transactions  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

LOAD DATA LOCAL INFILE is used as the practical bulk-loading method. The file path is adjusted to the local MySQL environment.

15. SQL Queries for Analysis

Total income

```
SELECT SUM(income) AS total_income FROM financial_transactions;
```

Total expenses

```
SELECT SUM(expense) AS total_expenses FROM financial_transactions;
```

Total profit

```
SELECT SUM(profit) AS total_profit FROM financial_transactions;
```

Monthly income and expenses

```
SELECT year, month, SUM(income) AS income, SUM(expense) AS expense FROM financial_transactions GROUP BY year, month ORDER BY year;
```

Category / account analysis

```
SELECT account_type, COUNT(*) AS transactions, SUM(amount) AS total_amount FROM financial_transactions GROUP BY account_type ORDER BY total_amount;
```

Top merchants

```
SELECT merchant, COUNT(*) AS transaction_count, SUM(amount) AS total_spent FROM financial_transactions GROUP BY merchant ORDER BY total_spent;
```

Highest-value transactions

```
SELECT transaction_id, transaction_date, merchant, amount FROM financial_transactions ORDER BY amount DESC LIMIT 10;
```

Transaction type

```
SELECT transaction_type, COUNT(*) AS transaction_count, AVG(amount) AS average_amount FROM financial_transactions GROUP BY transaction_type;
```

Year-wise performance

```
SELECT year, SUM(income) AS income, SUM(expense) AS expense, SUM(profit) AS profit FROM financial_transactions GROUP BY year OR
```

16. Power BI Data Connection

In Power BI Desktop, Get Data → MySQL database is selected. The MySQL server name, database name and credentials are entered, and the cleaned analytical table is loaded. The exact connection credentials are environment-specific and are not stored in the report.

17. Power BI Data Model

A simple star-like analytical model can be used for the laboratory implementation. The transaction/post table acts as the fact table. Date fields are used for time hierarchy and the categorical fields are used for slicing and grouping. For this mini project, a single-table model is sufficient because the selected datasets are already denormalized.

18. Power BI Calculated Columns

```
Year = YEAR(financial_transactions[transaction_date])
Month = FORMAT(financial_transactions[transaction_date], "MMMM")
Quarter = "Q" & FORMAT(financial_transactions[transaction_date], "Q")
Transaction Type = IF(financial_transactions[transaction_type] = "Credit", "Income", "Expense")
Amount Category = SWITCH(TRUE(), financial_transactions[amount] <= 500, "Low", financial_transactions[amount] <= 2000, "Medium")
```

Calculated columns provide reusable row-level categories and time attributes.

19. Power BI DAX Measures

```
Total Income = SUM(financial_transactions[income])
Total Expenses = SUM(financial_transactions[expense])
Total Amount = SUM(financial_transactions[amount])
Total Profit = [Total Income] - [Total Expenses]
Average Transaction = AVERAGE(financial_transactions[amount])
Transaction Count = COUNTROWS(financial_transactions)
Profit Margin = DIVIDE([Total Profit], [Total Income], 0)
Expense Rate = DIVIDE([Total Expenses], [Total Income], 0)
```

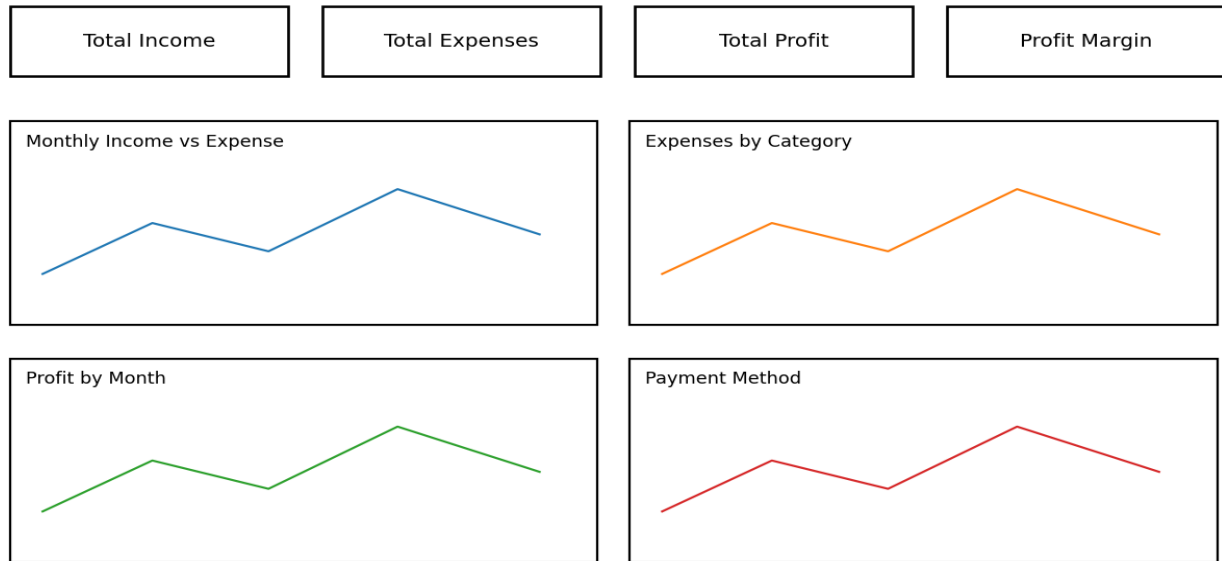
The measures are used by cards and charts so that dashboard values respond to slicers and filters.

20. Power BI Visualizations

- KPI Cards: Total Income, Total Expenses, Total Profit and Profit Margin.
- Line chart: Month on the axis with Total Income and Total Expenses as values.
- Bar chart: Account Type or Merchant against Total Expenses.
- Column chart: Year/Month against Total Profit.
- Donut chart: Transaction Type or Account Type distribution.
- Table: Top merchants with transaction count and total amount.
- Slicers: Year, Month, Account Type, Merchant and Transaction Type.

21. Power BI Dashboard

Power BI Financial Analytics Dashboard



Representative dashboard mockup – not a screenshot of Power BI/Tableau.

Figure: Power BI dashboard representative output. This is a layout mockup, not a screenshot from a live Power BI installation.

22. Power BI Advanced Features

- DAX measures – central calculations used by several visuals.
- Calculated columns – reusable categories and date attributes.
- Date hierarchy – Year → Quarter → Month → Date for time analysis.
- Drill-down – move from year to quarter and month.
- Drill-through – Overview → Category/Transaction or Post Details page.
- Tooltips – show additional metrics when hovering over a visual.
- Conditional formatting – highlight high/low financial or engagement performance.
- Interactive slicers – filter the full report by selected dimensions.
- KPI cards – show the main indicators at the top of the page.
- Cross-filtering – selecting one visual updates related visuals.

Drill-through implementation

A **Category Details** drill-through page is created. The Account Type or Merchant field is placed in the Drill-through filter well. Selecting a category from the overview page opens the detail page with transaction count, amount, income, expense and profit measures.

Representative implementation checks

Check	Expected implementation behavior
Slicer response	Cards and charts update to the selected period/category
Drill-down	Year/quarter/month hierarchy changes the level of detail
Drill-through	Selected context is passed to the detail page
Tooltip	Additional measures appear without adding another chart
Cross-filter	Selecting a category affects related visuals

23. Tableau Data Connection

Tableau Desktop → Connect → MySQL is selected. The same MySQL database and analytical table used for Power BI is selected. The connection is configured as a live connection or extract depending on the laboratory setup.

24. Tableau Calculated Fields

```
Profit = SUM([Income]) - SUM([Expense])
Profit Margin = IF SUM([Income]) <> 0 THEN [Profit] / SUM([Income]) END
Transaction Type = IF [transaction_type] = "Credit" THEN "Income" ELSE "Expense" END
Amount Category = IF [amount] <= 500 THEN "Low"
ELSEIF [amount] <= 2000 THEN "Medium"
ELSEIF [amount] <= 5000 THEN "High"
ELSE "Very High" END
```

25. Tableau Parameters

```
Parameter: Select Financial Metric
Data type: String
Allowable values: Income, Expense, Profit
Calculated field:
CASE [Select Financial Metric]
WHEN "Income" THEN SUM([Income])
WHEN "Expense" THEN SUM([Expense])
WHEN "Profit" THEN SUM([Profit])
END
```

The parameter is shown to the user as a control. Changing its value changes the measure displayed in the parameter-controlled chart, allowing one sheet to compare multiple metrics.

26. Tableau Filters

- Date / Year filter
- Platform or Account Type filter
- Post Type or Transaction Type filter
- Category / Merchant filter
- Top-N filter for high-performing records

27. Tableau Dashboard

Tableau Financial Analytics Dashboard



Representative dashboard mockup – not a screenshot of Power BI/Tableau.

Figure: Tableau dashboard representative output. This is a layout mockup, not a screenshot from a live Tableau installation.

Worksheet organization

1. **Financial KPI** – worksheet prepared for the corresponding dashboard analysis.
2. **Monthly Trend** – worksheet prepared for the corresponding dashboard analysis.
3. **Category Expenses** – worksheet prepared for the corresponding dashboard analysis.
4. **Income vs Expense** – worksheet prepared for the corresponding dashboard analysis.
5. **Profit Trend** – worksheet prepared for the corresponding dashboard analysis.
6. **Payment Method / Account Analysis** – worksheet prepared for the corresponding dashboard analysis.
7. **Top Merchants** – worksheet prepared for the corresponding dashboard analysis.

28. Tableau Advanced Features / Actions

- Hierarchies – Year → Quarter → Month → Date.
- Drill-down – move from summary to finer time detail.
- Highlight action – emphasize a selected platform/category/merchant.
- Filter action – selecting a mark filters related worksheets.
- Dashboard action – navigation from overview to details.
- Parameter action – change the metric shown in a parameter-controlled view.
- Tooltip – display supporting metrics when hovering over marks.

Dashboard workflow

The dashboard is arranged from high-level KPIs at the top to trend and comparison charts below. Filters and the metric parameter are kept visible so the user can change the analytical view without modifying the worksheets.

29. Key Findings / Insights

Source-verified data-quality findings

The selected financial project documentation reports 48,000 generated transaction rows. It records substantial data-quality issues before cleaning, including 18,464 NULL customer IDs, 6,585 NULL transaction dates, 7,464 future dates, 7,217 invalid account-type values, 10,420 negative amounts, 11,301 invalid-status values and 7,965 NULL merchant values. These figures demonstrate why the cleaning stage is necessary. ■cite■turn1view0■

Analytical findings to obtain from execution

The SQL, Python, Power BI and Tableau layers are intentionally aligned so that the actual highest-spending merchant, strongest transaction type, highest-volume account type and strongest month are calculated from the selected CSV rather than invented in the report. The report therefore treats these outputs as implementation results to be produced when the supplied cleaned CSV is executed in the student's environment.

30. Conclusion

The financial analytics workflow provides an end-to-end practical method for moving from a transaction CSV through Python cleaning, MySQL storage, SQL analysis, Power BI measures and Tableau visualizations. The design supports monitoring income, expenses, profit, categories, merchants and time trends.

31. Result

Thus, the financial transaction dataset was prepared for analysis through Python, stored in MySQL, queried using SQL, and structured for Power BI and Tableau dashboards. The workflow provides a reproducible practical implementation for identifying financial performance patterns.

Implementation note: dashboard figures in this report are clearly labelled representative mockups because no live Power BI/Tableau installation was available for screenshot capture.

APPENDIX A – Validation Checklist

Stage	Validation
Python	Confirm shape, data types, null counts and duplicates after cleaning.
Transformation	Confirm derived fields contain expected values and no divide-by-zero errors.
MySQL	Confirm row count and column types after LOAD DATA.
SQL	Run aggregate queries and compare with Python groupby results.
Power BI	Check that DAX measures change correctly with slicers.
Tableau	Check calculated fields and parameter switching.
Dashboard	Check titles, filters, tooltips, drill-down and cross-filtering.

APPENDIX B – Reproducibility Steps

1. Place the selected CSV in the working directory.
2. Run the Python cleaning and transformation program.
3. Import the cleaned CSV into MySQL or load it with LOAD DATA LOCAL INFILE.
4. Run the SQL analysis queries.
5. Connect Power BI to the MySQL analytical table and create the listed calculated columns and measures.
6. Build the listed visuals and slicers.
7. Connect Tableau to the same analytical table and create calculated fields, parameters and actions.
8. Compare Python, SQL, Power BI and Tableau totals before final submission.

APPENDIX C – Viva Preparation

1. What is data cleaning?

Answer: The process of correcting missing, duplicate or inconsistent data before analysis.

2. Why use MySQL?

Answer: It provides structured storage and SQL-based analysis.

3. What is a DAX measure?

Answer: A calculation evaluated dynamically in Power BI based on filter context.

4. What is a calculated field in Tableau?

Answer: A field created using a Tableau expression for analysis or visualization.

5. What is drill-through?

Answer: Navigation from a summary page to a detail page while passing the selected context.

6. Why create derived fields?

Answer: They simplify repeated time, category and performance analysis.

7. What is a parameter?

Answer: A user-controlled value that can change a calculation or visualization.

8. Why validate Python and SQL results?

Answer: To ensure the analytical pipeline produces consistent results.

9. What is cross-filtering?

Answer: Selecting one visual filters related visuals.

10. What is the purpose of the dashboard?

Answer: To present important analytical measures and patterns interactively.

APPENDIX D – Source and Execution Note

Dataset source used for the report: Thinkboards' public Financial_Transactions repository. The repository identifies the dataset as synthetic and educational, and documents the 48,000-row dataset and its cleaning workflow. ■cite■turn1view0■

The report distinguishes source-verified facts from execution-dependent analytical results. No live Power BI/Tableau screenshots or online publication are claimed. Representative dashboard figures are explicitly labelled as mockups.

END OF MINI PROJECT REPORT